

融合捷度约束的端到端强化学习运动规划方法

作者姓名¹

^a 某某大学/机构, 某某路, 某某市, 000000, 中国

Abstract

Keywords: 运动规划, 强化学习, 捷度约束, 端到端控制, CNC

1. 引言

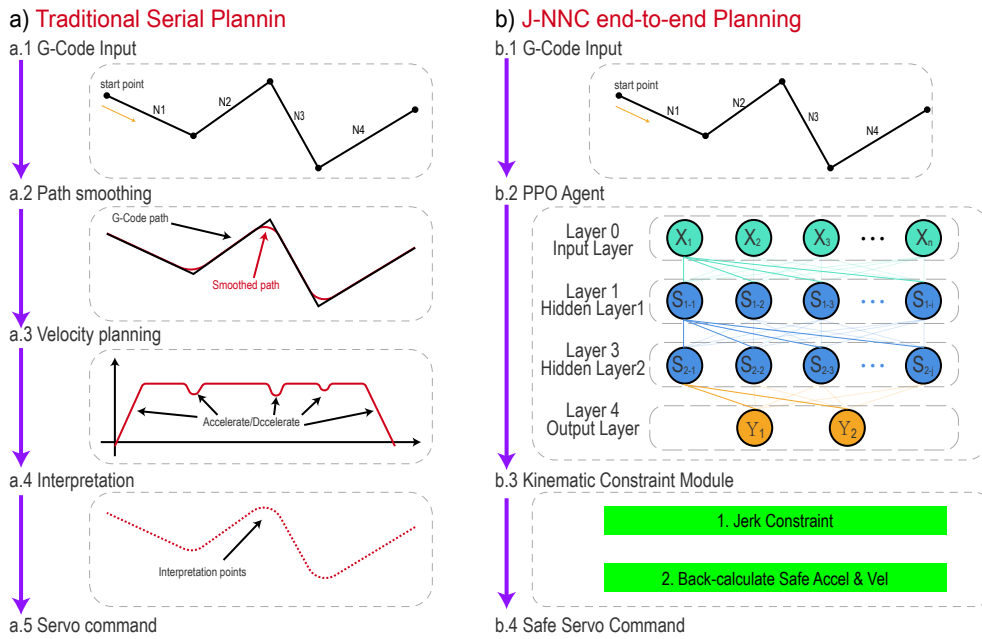


图 1: 传统串行式运动规划与 J-NNC 端到端规划的架构对比

J-NNC 通过周期性的神经网络推理，实时生成伺服指令（如图 1b 所示），因此被定义为一种“直接”运动规划方法。在此框架下，神经网络作为一个智能控制

单元，替代了传统 CNC 系统中独立的前瞻 (Look-ahead)、平滑 (Smoothing) 和轨迹规划等多个模块。与 NNC 相比，J-NNC 通过内置的捷度控制，确保输出的伺服指令直接满足运动学约束，从而无需后续的速度规划环节，实现了更彻底的端到端直接控制。

该方法的优势尤为体现在处理由长短线段混合组成的复杂刀具路径上，能有效平滑轨迹、优化速度与加速度，显著提升加工效率。其主要贡献在于模型的泛化能力：一旦训练完成，单个 J-NNC 模型即可鲁棒地适用于所有类型的刀具路径，无需针对特定路径进行迭代计算或重复训练。这种非迭代、实时的特性确保了 J-NNC 在工业应用中的可行性和高效性。



图 2: 捷度对运动平滑性影响的示意图

2. 问题阐述 (Problem Formulation)

2.1. 基于强化学习的运动规划框架

本文所提出的直接神经网络运动规划方法 (J-NNC) 旨在构建一个能够自主学习最优控制策略的智能体。该方法框架遵循强化学习 (RL) 的基本范式，将运动规划任务定义为智能体与环境的决策交互过程，如图 3 所示。

首先，基于输入的刀具路径和允差构建一个仿真环境。在每个决策步（等同于数控插补周期），神经网络智能体根据观测到的当前状态输出一个伺服指令作为动作，该动作需满足预设的捷度约束。环境在执行该动作后，会转移到下一状态并返回一个标量奖励。

图 3 占位符

图 3: J-NNC 的强化学习交互与训练框架

为了系统性地应用强化学习算法，将控制流问题建模为马尔可夫决策过程 (Markov Decision Process, MDP)。该过程可由一个元组 (S, A, R, P, γ) 定义：

- S 是所有可能状态组成的有限状态空间。
- A 是所有可能动作组成的有限动作空间。
- R 是奖励函数，表示在状态 s 下执行动作 a 后获得的即时奖励。
- P 是状态转移函数，定义了状态 s 执行动作 a 后，转移到下一状态 s' 的概率分布，即 $P(s'|s, a)$ 。
- γ (折扣因子 Discount Factor): 用于计算累积奖励，平衡即时奖励与未来奖励的重要性。

智能体的目标是学习一个最优策略 π ，以最大化从当前时刻开始的未来累积折扣奖励（即回报 G_t ）。

2.2. 刀具路径环境建模

为了应用强化学习解决刀具路径规划问题，构建刀具路径环境。中间的轨迹为 G 代码原始轨迹称为 P_{raw} ，可以用连续的点表示，如公式所示：

$$P_{raw} = \{p_1, p_2, \dots, p_N\} \quad (1)$$



图 4 占位符

图 4: 刀具路径可行域模型

式中 N 为 G 代码刀位点数。

直线刀具路径并不能包含加工中所需的所有信息，为了让智能体找到合理的运动规划方案，我们创建了一个可行域。通过轮廓允差 δ 对中心路径进行双向偏移，我们构建了由左边界和右边界包围的路径可行域（如图 4 所示）。

可行域的数学描述如下：首先，将 P_{raw} 向左偏置，偏置距离为允差的一半：

$$L_{left} = P_{raw} + \vec{n} \cdot \frac{\delta}{2} \quad (2)$$

我们将左偏移路径定义为路径 L_{left} 。同样的，我们可以得到右偏移路径：

$$L_{right} = P_{raw} - \vec{n} \cdot \frac{\delta}{2} \quad (3)$$

因此，将公式结合，可以得到可行域 Ω 由 L_{left} 和 L_{right} 组成，即：

$$\Omega = \{p \mid dist(p, P_{raw}) \leq \frac{\delta}{2}\} \quad (4)$$

综上所述，刀具路径环境模型将 G 代码指令转化为一个结构化的、允许局部优化的几何空间。

3. J-NNC 模型设计与实现

J-NNC 方法将对捷度 (Jerk) 约束直接整合到神经网络控制器中。本章节将详细阐述为该强化学习任务所设计的环境、动作空间、状态空间及奖励函数，并介绍神经网络模型的具体训练过程。

3.1. 捷度约束下的动作空间设计

借鉴直接控制方法 [?]，我们将 a_{raw} 定义为智能体期望的线速度变化量和角速度变化量，即 $a_{raw} = [l', \theta']$ ，这代表了高层级的“运动意图”。这个原始动作随后被送入一个确定性的 **运动学约束模块 (Kinematic Constraint Module, KCM)**，该模块负责将抽象意图转化为一个 100% 满足所有物理约束的最终动作 a_{final} ，并最终在环境中执行。



图 5: J-NNC 捷度约束下的动作空间设计与运动状态更新流程: (a) “决策-执行”分离的整体架构; (b) KCM 内部的“约束与反算”逻辑

KCM 的核心算法确保了输出动作的物理合理性，其流程如 Algorithm ?? 所示。该算法分为两个关键阶段：

1. **第一阶段：自顶向下 (Top-down) 施加约束。**算法首先根据 a_{raw} 计算出期望的目标速度，然后依次通过速度、加速度和捷度的约束层。在此级联约束中，捷度具有最高优先级。
2. **第二阶段：自底向上 (Bottom-up) 反算状态。**模块以第一阶段得到的 $jerk_{final}$ 为起点，利用运动学方程反向推算出在本周期内实际能达到的最终加速度和最终速度。最终的伺服步长指令便由这个合理的 v_t 确定。

通过这种自顶向下施加约束、再自底向上反算状态的机制，KCM 确保了输出的 a_{final} 在逻辑上是连贯的，并且同时满足了所有层级的运动学约束。

3.2. 状态空间设计

状态空间的设计构建了一个 12 维的特征向量，如图 6 所示：

Algorithm 1 运动学约束模块 (KCM)

Require: $a_{\text{raw}} = [l', \theta']$ {神经网络输出的原始动作 (意图)}

Require: $state_{t-1} = (v_{t-1}, acc_{t-1}, \dots)$ {上一时刻的完整运动学状态}

Ensure: $a_{\text{final}} = [length_{\text{prime}}, \theta_{\text{prime}}]$ {最终可执行动作}

Ensure: $state_t = (v_t, acc_t, jerk_t, \dots)$ {更新后的运动学状态}

- 1: // 1. 自顶向下 (Top-down) 施加约束, 捷度优先
 - 2: $v_{\text{target}} \leftarrow v_{t-1} + l'$
 - 3: $v_{\text{clipped}} \leftarrow \text{clip}(v_{\text{target}}, -V_{\text{max}}, V_{\text{max}})$
 - 4: $acc_{\text{needed}} \leftarrow \frac{v_{\text{clipped}} - v_{t-1}}{\Delta t}$
 - 5: $acc_{\text{clipped}} \leftarrow \text{clip}(acc_{\text{needed}}, -A_{\text{max}}, A_{\text{max}})$
 - 6: $jerk_{\text{needed}} \leftarrow \frac{acc_{\text{clipped}} - acc_{t-1}}{\Delta t}$
 - 7: $jerk_{\text{final}} \leftarrow \text{clip}(jerk_{\text{needed}}, -J_{\text{max}}, J_{\text{max}})$
 - 8: // 2. 自底向上 (Bottom-up) 反算, 确保状态合理
 - 9: $acc_t \leftarrow acc_{t-1} + jerk_{\text{final}} \cdot \Delta t$
 - 10: $v_t \leftarrow v_{t-1} + acc_t \cdot \Delta t$
 - 11: // 3. 生成最终动作
 - 12: $length_{\text{prime}} \leftarrow v_t$
 - 13: (角速度部分 θ_{prime} 的计算类似)
 - 14: $a_{\text{final}} \leftarrow [length_{\text{prime}}, \theta_{\text{prime}}]$
 - 15: **return** $a_{\text{final}}, state_t$
-

图 6 占位符

图 6: 状态空间的设计

- **运动学状态 (6 维):**
 - 线性运动学 (3 维): 包括当前时刻的线速度、线加速度和线捷度。
 - 角运动学 (3 维): 包括当前时刻的角速度、角加速度和角捷度。
- **路径信息 (3 维):**
 - 方向偏差角: 表示当前运动方向与参考路径切线方向的夹角。
 - 到下一拐点距离。
 - 下一拐点转角: 量化下一个路径点的转弯剧烈程度。
- **任务进度与历史动作 (3 维):** 该模块提供任务的整体进展和上一时刻的决策。
 - 路径总进度: 表示当前位置在整个路径上的完成度。
 - 上一时刻最终动作: 上一时刻经过运动学约束模块 (KCM) 处理后最终执行的角速度和线速度。

3.3. 奖励函数设计

在任务中，期望在直线段走得很快并且准确识别弯道，根据弯道选择合理的速度和路径通过，因此设计了如下奖励：

- **效率奖励:** 我们将其设计为与智能体沿路径切向速度成正比，即 $r_e \propto v_t$ 。
- **精度奖励:** 使用轮廓误差的二次方作为惩罚项，即 $r_c \propto -error^2$ 。
- **平滑性奖励:** 对瞬时捷度的绝对值施加惩罚，即 $r_j \propto -|jerk|$ 。

为此，我们将总奖励设计为以下几个子项的加权和：

$$R = w_e r_e + w_c r_c + w_j r_j \quad (5)$$

3.4. PPO 算法

J-NNC 智能体基于近端策略优化 (PPO) 算法实现, 并采用了一个标准的 Actor-Critic (演员-评论家) 架构。其中, 策略网络 (Actor) 与价值网络 (Critic) 共享一个由三层全连接层 (激活函数为 ELU) 组成的共同特征提取主干。Actor 网络设计为两个独立的输出头, 分别用于输出指令速度和指令方向; 而 Critic 网络则为单个输出头, 用于评估状态价值。详细的超参数配置在表 1 中列出。

4. 实验与结果 (Experiment & Results)

4.1. 实验方案与平台设计

4.1.1. 实验环境与平台

仿真平台: 我们搭建了一个基于 Python 3.8 与 PyTorch 1.10 的仿真实验平台。该平台能够精确模拟 CNC 机床的运动学过程, 并为强化学习智能体的训练与评估提供支持。智能体的训练借助了 Stable-Baselines3 强化学习库来实现 PPO 算法。所有仿真实验均在一台配备有 Intel Core i7-12900K CPU 和 NVIDIA RTX 3090 GPU 的服务器上完成。整个训练过程包含 $1e6$ 个总时间步 (timesteps)。为了保证研究的可复现性, 所有关键的超参数设置均遵循了 PPO 算法的标准实践, 具体数值如表 ?? 所示。

物理平台: 为了最终验证算法的实际工程价值, 我们还将在一台 **VMC-850 型三轴立式加工中心** 上进行物理加工实验。物理实验将使用 6061 铝合金作为加工材料, 其结果将在后续章节中进行详细分析。

4.2. 对比基线方法

为全面评估 J-NNC 的性能, 我们选取了以下两种具有代表性的方法作为基线进行对比:

- **传统串行方法 (Traditional Method):** 该方法代表了工业界广泛采用的技术路线, 即先使用 B 样条对路径进行几何平滑, 再采用 S 形加减速算法进行速度规划。

表 1: J-NNC 模型训练超参数设置

参数名称	值
状态维度	12
隐藏层维度	512
动作维度	2
Dropout 比率	0.1
策略网络学习率	1e-5 (0.00001)
价值网络学习率	5e-5 (0.00005)
折扣因子	0.99
GAE Lambda	0.95
PPO 裁剪系数	0.1
训练轮次	10
学习率衰减因子	0.5
Actor 耐心值	10
Critic 耐心值	5
最大梯度范数	0.5

- **NNC (Neural Network Controller)**: 该方法是 Li 等人提出的端到端直接运动规划方法 [?]，它同样基于强化学习，但其动作空间未包含显式的捷度硬约束。

4.3. 性能评价指标与测试路径

我们从“效率、精度、平滑性”三个维度，采用以下四个关键性能指标（KPIs）对结果进行量化评估：1) 加工时间 (Processing Time, s): 衡量加工效率；2) 最大轮廓误差 (Max Contour Error, μm): 衡量最差情况下的加工精度；3) 平均轮廓误差 (Mean Contour Error, μm): 衡量整体加工精度；4) 最大捷度 (Max Jerk, m/s^3): 衡量运动平滑性。

为测试模型的泛化能力，我们选取了两种几何特征的典型刀具路径进行实验，如图 7 所示：1) S 形路径，代表了平滑曲线加工场景；2) 蝶形路径，其包含了大量

的急转弯和曲率突变，对规划器的性能提出了严峻挑战。



图 7 占位符

图 7: 实验用典型刀具路径

4.4. 对比实验与分析

4.4.1. 运动学曲线的定性比较

运动学曲线的平滑程度是评价运动规划质量最直观的方式。我们首先在 S 形路径上对三种方法的运动学曲线进行了对比，结果如图 8 所示。



图 8 占位符

图 8: 简单路径下的性能验证：S 形路径

从图 8 可以清晰地观察到，三种方法在运动学特性上表现出显著差异：

- **速度曲线分析：**J-NNC（绿色实线）展现了最为平滑的变速过程，其速度曲线呈自然的 S 形。传统方法（蓝色虚线）则呈现出典型的梯形速度包络。而 NNC（橙色点划线）虽然也试图平滑，但在变速的起始和结束阶段仍存在较明显的波动。
- **加速度曲线分析：**J-NNC 的加速度曲线连续且光滑。相比之下，NNC 的加速度曲线虽然连续，但存在一定程度的高频振荡。传统方法的加速度曲线则呈现为不连续的梯形波，其拐点处的瞬时突变意味着理论上无穷大的捷度。
- **捷度曲线分析：**在捷度曲线中，J-NNC 的优越性体现得最为明显。其捷度值被严格约束在预设的物理极限（灰色虚线带）内，全程平稳可控。NNC 由于缺乏硬约束，其捷度出现了显著的超调和尖峰。传统方法则在加速度变化的时刻产生了剧烈的脉冲式捷度。

4.4.2. 泛化能力的可视化分析

为进一步展示 J-NNC 的泛化能力和智能决策行为，我们在蝶形路径上对其生成的轨迹进行了速度热力图可视化，如图 9 所示。



图 9: 复杂路径下的综合性能分析：蝶形路径

图 9a 中的速度热力图清晰地揭示了 J-NNC 智能体无需任何额外指令或预处理，便自主学会了最优的全局加减速策略。图 9b 则展示了在蝶形路径这一严苛挑

战下的运动学曲线。相比 S 形路径，蝶形路径包含的连续急转弯对规划器的稳定性提出了更高要求。图中可见，J-NNC 依然保持了极度平滑且严格受控的加减速和捷度曲线。

4.4.3. 性能指标的量化对比

为了更精确地评估各方法的综合性能，我们在两种测试路径上对上述四个关键性能指标进行了量化统计，详细数据如表 ?? 所示。

表 2: 不同方法在两种典型路径下的性能量化对比

路径类型	性能指标 (Metric)	J-NNC (本文)	NNC (基线 1)	传统方法 (基线 2)
S 形路径	加工时间 (s)			
	最大轮廓误差 (μm)			
	平均轮廓误差 (μm)			
	最大捷度 (m/s^3)			
蝶形路径	加工时间 (s)			
	最大轮廓误差 (μm)			
	平均轮廓误差 (μm)			
	最大捷度 (m/s^3)			

从表 2 的量化数据可以得出以下核心结论：

- 平顺性最优，且优势巨大：**在两种路径下，J-NNC 的最大捷度均被严格控制在预设的物理极限内，其数值相较于 NNC 降低了一个数量级以上，更是远优于传统方法。
- 效率与精度实现更优平衡：**J-NNC 在实现极致平顺性的同时，并未牺牲效率和精度。其加工时间相比传统方法有显著缩短，且轮廓误差也保持在与 NNC 相当甚至更优的极低水平。
- 复杂任务下的鲁棒性强：**在几何形状更为复杂的蝶形路径上，传统方法为了保证精度不得不大幅牺牲速度，而 NNC 的捷度指标也出现了恶化。相比之下，J-NNC 的各项性能指标依然全面领先。

4.5. 消融实验 (Ablation Study)

为了验证 J-NNC 模型中各个创新设计的有效性，我们进行了一系列消融实验。我们以最具挑战性的蝶形路径为测试对象，通过“移除”模型中的关键组件来观察其性能变化。实验的量化结果如表 ?? 所示，其可视化分析如图 10 所示。

表 3: J-NNC 模型关键组件的消融实验分析 (以蝶形路径为例)

模型配置 (Model Configuration)	加工时间 (s)	最大轮廓误差 (μm)	最大捷度 (m/s^3)
完整 J-NNC 模型 (本文)	XX.XX	XX.XX	XX.XX
J-NNC (无 KCM 模块)	任务失败	任务失败	任务失败
J-NNC (无捷度奖励 r_j)	XX.XX	XX.XX	XX.XX



图 10: 消融实验的可视化分析

从表 3 和图 10 中，我们可以得到两个关键结论：

- KCM 模块的不可或缺性：**从表 3 可见，当移除运动学约束模块（KCM）后，模型完全无法完成任务。这强有力地证明了 KCM 是整个方法得以成功训练和运行的“安全基石”。
- 捷度奖励 r_j 的精确引导作用：**当仅移除奖励函数中的捷度惩罚项 r_j 后，模型虽然仍能在 KCM 的保护下完成任务，但其性能出现了显著退化。图 10b 显示，没有 r_j 引导的策略（紫色虚线）在极限边界附近表现出剧烈的、高频的震荡。而完整模型则平滑得多。

4.6. KCM 模块内部机制探索

为了进一步探究 KCM 模块是如何智能地工作的，我们对其“干预度”进行了可视化，并将其与轮廓误差进行关联分析，如图 11 所示。



图 11: KCM 干预度与轮廓误差的关联分析

图 11 揭示了 KCM 模块“按需介入、精准调控”的智能特性。在路径平直区域，KCM 几乎“放手”让智能体自由探索。然而，当路径进入急转弯处，轮廓误差有增大的趋势，此时 KCM 的干预度瞬时飙升，对神经网络输出的“激进”动作进行了强力修正。

4.7. 物理加工实验验证

为了最终验证 J-NNC 方法在仿真环境中体现出的平顺性优势能否转化为实际的工程价值，我们在一台 VMC-850 型三轴立式加工中心上进行了物理加工对比实验。我们选用 6061 铝合金作为加工材料，使用相同的球头铣刀与切削参数，分别采用机床自带的传统控制器和由 J-NNC 模型生成的运动指令，加工了两个完全相同的“蝶形”路径工件。

加工结果对比如图 12 所示。

从宏观对比中，可以初步观察到 J-NNC 加工的工件表面反光更为均匀。如图 12c 所示，传统方法由于其固有的加速度不连续性，在路径曲率变化剧烈的拐角处产生了明显的机械振动，导致加工表面留下了清晰可见的振纹 (chatter marks) 和不均匀的刀痕。相比之下，如图 12d 所示，采用 J-NNC 方法加工的同一位置，表面则异常光滑平整，刀痕均匀、细腻且连续。



图 12 占位符

图 12: J-NNC 与传统方法在蝶形路径下的实物加工表面质量对比

该物理实验结果强有力地证明了：J-NNC 不仅在仿真数据上表现优异，其算法优势更能直接转化为可观的、实际的加工质量提升。这最终确立了本方法在精密制造领域的工程应用价值。

5. 结论

本文提出的 J-NNC 方法通过引入 KCM 模块和捷度约束的强化学习框架，成功解决了端到端运动规划中的平滑性难题。仿真与物理实验均证明，该方法在保证效率和精度的同时，极大地提升了运动的平顺性，具有重要的工程应用价值。